

Universidad del Valle de Guatemala

Facultad de Ingeniería

Departamento de Ciencias de la Computación

Construcción de Compiladores

[M2] Fase de Compilación: Generación de Código Intermedio

Generador de código de tres direcciones de Compiscript

ANTLR 4.13.2 • C++17 • TAC • IDE web

`https://github.com/G1LB3TO/Proyecto1_Compis_Generador_Analizador_Lexico/tree/Compiladores2-Proyecto2`

Catedrático

Ing. Pablo Koch

Integrantes

Luis Gonzalez — 23353

Joel Jaquez — 23369

Guatemala, 18 de octubre de 2026

Índice

1. Introducción **3**

2. Objetivos **3**

2.1. Objetivo general 3

2.2. Objetivos específicos 3

3. Arquitectura general **4**

3.1. El pipeline 4

3.2. Dos Visitors con responsabilidades distintas 4

3.3. Núcleo independiente de la interfaz 5

3.4. Cuándo se genera código 5

4. Estructura del repositorio y trazabilidad por commits **5**

4.1. Historial de la rama 6

4.2. Lectura del historial 7

5. Descripción de los archivos por capa **7**

5.1. Núcleo del compilador 7

5.2. Pruebas, IDE y documentación 8

6. Diseño del lenguaje intermedio **8**

6.1. Representación: cuádruplos 8

6.2. Direcciones y nombres 8

6.3. Conjunto de instrucciones 9

7. Traducción de Compiscript a TAC **10**

7.1. Expresiones aritméticas, precedencia y asociatividad 10

7.2. Expresiones lógicas y comparaciones 11

7.3. Condicionales 11

7.4. Ciclos 11

7.4.1. break y continue 12

7.5. switch 12

7.6. Funciones, parámetros, retorno y recursividad 13

7.7. Funciones anidadas y closures 13

7.8. Clases, objetos y métodos 14

7.9. Listas y estructuras de datos 14

7.10. try/catch 14

7.11. Ejemplo integral 15

8. Manejo de temporales **17**

9. Tabla de símbolos y entornos de ejecución **18**

9.1. Información agregada para la generación de código 18

9.2. Supuestos de tamaño y alineación 18

9.3. Registros de activación 19

9.4. Acceso a datos no locales 20

9.5. Interacción con las fases del compilador 20

10. El IDE **20**

10.1. Pestaña Código TAC 20

10.2. Closures en el IDE 23

10.3. Tabla de símbolos y ámbitos extendidos 24

10.4. Reporte de errores 25

11. Batería de pruebas **26**

11.1. Escenarios de generación de código 27

12. Compilación y ejecución **28**

12.1. Requisitos 28

12.2. Construcción 28

12.3. Ejecución del IDE 29

12.4. Uso directo del CLI 29

12.5. Verificación realizada para este informe 29

13. Decisiones, supuestos y limitaciones **29**

14. Correspondencia con la rúbrica **30**

15. Conclusiones **31**

16. Referencias **31**

1. Introducción

Este informe documenta la fase de **generación de código intermedio** del compilador de **Compiscript**, un lenguaje basado en un subconjunto de TypeScript. El trabajo parte de lo construido en la fase anterior —el analizador sintáctico y semántico con su tabla de símbolos— y agrega una segunda pasada que recorre el mismo árbol sintáctico y lo traduce a **código de tres direcciones** (TAC, por *three-address code*), la representación que servirá de base para una posterior generación de código ensamblador u objeto.

Como en la fase anterior, **todo el compilador está escrito en C++17**. El generador de código es un segundo *Visitor* de ANTLR, el `TacGenerator`, que se enlaza en la misma biblioteca nativa que el analizador semántico. Python aparece únicamente como adaptador web (el servidor Flask que sirve el IDE) y JavaScript únicamente para presentar resultados: ninguna regla de traducción está implementada fuera de C++.

La fase aporta tres piezas, que corresponden a los componentes evaluados:

- **Un lenguaje intermedio documentado**: cuádruplos con operación, dos argumentos y un resultado, con una forma textual legible y un conjunto de instrucciones para datos, control de flujo, funciones, closures, clases y listas.
- **Un generador de TAC** que traduce expresiones, declaraciones, `if/else`, `while`, `do-while`, `for`, `foreach`, `switch`, `break`, `continue`, funciones recursivas y anidadas, clases, objetos y listas, con un algoritmo de asignación y reciclaje de variables temporales.
- **Una tabla de símbolos extendida** con tamaños, desplazamientos, clase de almacenamiento, etiquetas y profundidad léxica, más el diseño abstracto de los **registros de activación** de cada función.

El sistema acepta archivos `.cps` y produce, en un único documento JSON, los tokens, el árbol sintáctico, los diagnósticos, la tabla de símbolos con la información de generación de código, el texto TAC, los cuádruplos y los registros de activación. La batería de pruebas pasa de 95 a **107 escenarios** y todos aprueban.

2. Objetivos

2.1. Objetivo general

Implementar un generador de código intermedio para Compiscript que, a partir del árbol sintáctico y la tabla de símbolos producidos por el análisis semántico, emita código de tres direcciones correcto y documentado, manejando temporales, entornos de ejecución y registros de activación.

2.2. Objetivos específicos

- Diseñar y documentar un lenguaje intermedio de tres direcciones, con sintaxis consistente y ejemplos de traducción.
- Traducir expresiones aritméticas, lógicas y de comparación respetando precedencia y asociatividad, y las declaraciones y asignaciones de variables y constantes.
- Traducir el control de flujo (`if/else`, `while`, `do-while`, `for`, `foreach`, `switch`, `break` y `continue`) mediante etiquetas y saltos.

- Traducir funciones, paso de parámetros, retorno, recursividad, funciones anidadas y closures, así como clases, objetos, métodos, atributos y listas.
- Implementar un algoritmo de asignación y reciclaje de temporales que minimice la cantidad de temporales vivos.
- Extender la tabla de símbolos con direcciones, desplazamientos, tamaños y etiquetas, y modelar registros de activación.
- Ampliar la batería de pruebas y el IDE para ejecutar y mostrar la generación de código intermedio.

3. Arquitectura general

3.1. El pipeline

El flujo de información va del texto fuente al documento JSON que consume el IDE. Cada etapa tiene una responsabilidad única y un artefacto observable. Las dos etapas nuevas de esta fase son el `TacGenerator` y la extensión de la tabla de símbolos:

```
Código .cps
    |
    v
CompiscriptLexer  (ANTLR, C++)
    | tokens
    v
CompiscriptParser (ANTLR, C++)
    | parse tree ---------> tree_serializer -> árbol JSON -> IDE
    v
SemanticAnalyzer  (Visitor C++, primera pasada)
    |-- tipos, ámbitos, funciones, clases, listas
    |-- tabla de símbolos + SemanticBindings
    v
(solo si no hay errores léxicos, sintácticos ni semánticos)
    v
TacGenerator      (Visitor C++, segunda pasada)  <<< NUEVO
    |-- layoutSymbols: tamaños, offsets, etiquetas, frames
    |-- traducción a cuádruplos con pool de temporales
    v
JSON: diagnósticos + símbolos + TAC + registros de activación
    |
    v
Flask (adaptador) -> IDE web (pestaña "Código TAC")
```

3.2. Dos Visitors con responsabilidades distintas

`SemanticAnalyzer` y `TacGenerator` son Visitors diferentes. El primero decide qué significa cada identificador, comprueba tipos y construye los ámbitos. El segundo consume esa información y vuelve a recorrer el mismo árbol para emitir instrucciones. Separarlos evita duplicar la resolución de nombres y garantiza que el generador nunca intente corregir ni reinterpretar un programa inválido.

El puente entre ambos es la estructura `SemanticBindings`, que el analizador llena mientras recorre el árbol. Asocia cada nodo del *parse tree* con tres cosas:

- `symbols`: el identificador del símbolo al que se refiere un nodo (una declaración, un uso, una llamada, un `new`);
- `scopes`: el ámbito que abre un nodo (programa, bloque, función, clase, `for`, `foreach`, `catch`);
- `expressions`: el tipo sintetizado de cada expresión.

Estas asociaciones son efímeras: viven solo mientras `analyzeSource` conserva el árbol de ANTLR. Gracias a ellas el generador no resuelve nombres por texto, sino por identificador de símbolo; el sufijo `#id` que aparece en el TAC (por ejemplo `@x#2`) es ese identificador, y es lo que evita que dos variables sombreadas compartan una misma dirección.

3.3. Núcleo independiente de la interfaz

El núcleo C++ sigue exponiendo una sola función pública, `analyzeSource`, que devuelve un `AnalysisResult`. El CLI, la batería de pruebas y el servidor Flask la usan por igual, de modo que lo que prueba la batería es exactamente lo que ejecuta el IDE. El resultado ahora incluye el `tac` (`TacResult`) y el indicador `tac_ok`.

3.4. Cuándo se genera código

El generador **solo se ejecuta cuando las fases anteriores terminaron sin errores**. Si falla una fase, las posteriores se marcan como omitidas y el resultado explica el motivo:

Entrada	Sintaxis	Semántica	TAC
Error léxico o sintáctico	con errores	omitida	omitido
Error semántico	correcta	con errores	omitido
Programa válido	correcta	correcta	generado

Tabla 1: Estado de cada fase según el punto en que falla el programa.

Así nunca se produce código parcial para programas rechazados. Los tokens, el árbol y los diagnósticos se siguen enviando al IDE, porque son la información que el usuario necesita para corregir la entrada.

4. Estructura del repositorio y trazabilidad por commits

El desarrollo de esta fase se realizó en la rama `Compiladores2-Proyecto2`, a partir del último commit de la fase anterior (`c8fdf9a`, informe del análisis semántico). Cada commit corresponde a una unidad funcional completa y a un solo autor, de modo que la contribución individual queda registrada en el historial, como exige el enunciado. Los seis commits se publicaron el 17 de octubre de 2026 y suman 18 archivos modificados, 2 356 líneas agregadas y 59 eliminadas. Las identidades de Git corresponden a los dos integrantes:

Identidad en Git	Correo	Integrante
`G1LB3T0`	luis.yo09@hotmail.com	Luis Gonzalez (23353)
`jaq23369`	jaq23369@uvg.edu.gt	Joel Jaquez (23369)

Tabla 2: Identidades de Git utilizadas en la rama.

4.1. Historial de la rama

La tabla siguiente recorre los seis commits de la fase en orden cronológico y explica qué archivos introduce cada uno y por qué. Es, en la práctica, el mapa de lectura del repositorio.

Commit	Autor	Aporte y archivos
`ca76f1e`	Luis Gonzalez	**[ADD] Extender modelo semántico para generación TAC.** Prepara el terreno para la segunda pasada. En `model.h` agrega a `Symbol` los campos `size`, `offset`, `lexical_depth`, `storage` y `label`; a `Scope`, `lexical_depth` y `frame_size`; y las estructuras `TacInstruction`, `ActivationRecord`, `TacResult` y `AnalysisResult.tac`. En `semantic_analyzer.{h,cpp}` introduce `SemanticBindings` y registra en él símbolos, ámbitos y tipos de expresión. 3 archivos, +124/−10.
`84ae937`	Joel Jaquez	**[ADD] Implementar generador TAC de Compiscript.** El corazón de la fase. Crea `tac_generator.h` (184 líneas) y `tac_generator.cpp` (1 211 líneas): el Visitor que traduce cada construcción, el cálculo de *layout* de símbolos y frames, el pool de temporales y el formateo textual de los cuádruplos. Registra el nuevo fuente en `CMakeLists.txt`. 3 archivos, +1 397/−1.
`6c501f3`	Luis Gonzalez	**[ADD] Integrar TAC en servicio y salida JSON.** Conecta el generador al pipeline. En `service.cpp` invoca al `TacGenerator` solo si la semántica fue correcta y marca el TAC como omitido en caso contrario. En `json.cpp` serializa `tac` (texto, cuádruplos, métricas de temporales y registros de activación) y los nuevos campos de símbolos y ámbitos. 3 archivos, +88/−4.
`9c0cf1c`	Joel Jaquez	**[ADD] Mostrar código intermedio en el IDE.** Agrega la pestaña *Código TAC* con métricas, texto copiable, tabla de cuádruplos y tabla de registros de activación; la tarjeta de resumen *Código TAC*; y las columnas de almacenamiento, *offset*, profundidad y tamaño de frame en las tablas de símbolos y ámbitos. Modifica `app.js`, `style.css` e `index.html`. 3 archivos, +118/−13.
`66f83c3`	Luis Gonzalez	**[TEST] Cubrir generación TAC de Compiscript.** Amplía `compiscript_tests.cpp` con 12 escenarios nuevos (de 95 a 107) que verifican la forma del TAC, el reciclaje de temporales, el *layout* y la omisión ante errores, y agrega el programa de demostración `codigo_intermedio.cps`. 2 archivos, +189/−2.

Commit	Autor	Aporte y archivos
32305b5	Joel Jaquez	**[DOC] Documentar arquitectura y lenguaje TAC.** Crea docs/codigo-intermedio-tac.md (309 líneas) con el diseño del lenguaje intermedio, ejemplos y supuestos; actualiza docs/arquitectura-compiscript.md y el README.md con la compilación, el uso del CLI y del IDE, y ajusta .gitignore. 4 archivos, +440/−29.

Tabla 3: Trazabilidad de la rama Compiladores2-Proyecto2.

4.2. Lectura del historial

El orden de los commits refleja el orden natural de construcción de la fase. Primero se amplía el modelo de datos y se deja al analizador semántico registrando lo que el generador necesitará (ca76f1e); luego se implementa el generador (84ae937), se conecta al servicio y a la salida JSON (6c501f3) y se muestra en la interfaz (9c0cf1c). Finalmente llegan la evidencia de que funciona (66f83c3) y la explicación de por qué está diseñado así (32305b5). La autoría se alterna entre los dos integrantes: tres commits cada uno.

5. Descripción de los archivos por capa

La fase no modifica la gramática ni el código generado por ANTLR: reutiliza el parser de la fase anterior. Las tablas siguientes listan los archivos nuevos o modificados, con su tamaño actual en líneas.

5.1. Núcleo del compilador

Archivo	Estado	Líneas	Función
src/tac_generator.cpp	nuevo	1211	El Visitor de la segunda pasada. Traduce a cuádruplos, calcula el *layout* y administra temporales.
include/.../tac_generator.h	nuevo	184	Declaración del Visitor, TacValue y el pool de temporales.
include/.../model.h	modificado	186	Symbol y Scope extendidos; TacInstruction, ActivationRecord y TacResult.
src/semantic_analyzer.cpp	modificado	1261	Registra los *bindings* nodo → símbolo, ámbito y tipo.
include/.../semantic_analyzer.h	modificado	183	Declara SemanticBindings.
src/service.cpp	modificado	91	Encadena lexer, parser, semántica y TAC.
src/json.cpp	modificado	294	Serializa TAC, frames y campos de símbolos.

Tabla 4: Archivos del núcleo C++ tocados en esta fase.

5.2. Pruebas, IDE y documentación

Archivo	Líneas	Función
`tests/compiscript_tests.cpp`	816	107 escenarios (12 de generación de TAC).
`examples/.../codigo_intermedio.cps`	25	Programa integral de demostración.
`frontend/static/js/app.js`	1096	Pestaña *Código TAC*, columnas nuevas de símbolos y ámbitos.
`frontend/static/css/style.css`	243	Estilos de métricas y bloque de código.
`frontend/templates/index.html`	103	Pestaña y tarjeta de resumen.
`docs/codigo-intermedio-tac.md`	309	Especificación del lenguaje intermedio.
`docs/arquitectura-compiscript.md`	181	Arquitectura actualizada.
`README.md`	297	Compilación, ejecución y uso.

Tabla 5: Archivos de pruebas, interfaz y documentación.

El servidor Flask no cambia: `/api/analyze` ejecuta `compiscript_cli` y reenvía el JSON, que ahora trae el bloque `tac`.

6. Diseño del lenguaje intermedio

6.1. Representación: cuádruplos

Cada instrucción es un cuádruplo `TacInstruction`. La implementación conserva los cuádruplos estructurados para las pruebas, el IDE y las fases futuras; el texto se produce a partir de ellos y **nunca se vuelve a analizar** para recuperar información.

Campo	Significado
`op`	Código estable de la operación (`add`, `if_false`, `call`, ...).
`arg1`, `arg2`	Operandos, cuando la forma de instrucción los necesita.
`result`	Destino, etiqueta o nombre principal.
`index`	Posición de la instrucción en la secuencia.
`line`, `column`	Origen aproximado en el archivo `.cps`.
`text`	Serialización legible del mismo cuádruplo.

Tabla 6: Campos de un cuádruplo.

6.2. Direcciones y nombres

La forma de un operando indica dónde vive su valor. El sufijo `#id` es el identificador del símbolo en la tabla.

Forma	Ejemplo	Uso
Constante	`4`, `true`, `"hola"`	Literales del programa.
Global	`@total#3`	Símbolo sin función propietaria.
Local o parámetro	`%valor#7`	Símbolo dentro de un registro de activación.
Temporal	`%t0`	Resultado intermedio generado por el compilador.
Captura	`env[%base#1]`	Valor obtenido por el enlace de acceso.
Receptor	`%this`	Objeto implícito de un método.
Etiqueta	`L_while_cond_2`	Destino simbólico de un salto.

Tabla 7: Formas de dirección del TAC.

Las etiquetas de salto siguen el patrón `L_<tipo>_<n>`, donde `<n>` es un contador global. Las etiquetas de función se forman con la clase propietaria (si existe), el nombre y el identificador del símbolo, por ejemplo `fn_Acumulador_sumar_4`. Así son únicas y legibles.

6.3. Conjunto de instrucciones

Datos y expresiones.

op	Forma textual	Semántica
`declare`	`declare x : T`	Reserva una dirección simbólica.
`assign`	`x = y`	Copia el valor de `y`.
`add sub mul div mod`	`t = x op y`	Operación aritmética o concatenación ya validada.
`neg`, `not`	`t = -x`, `t = !x`	Operación unaria.
`eq ne lt le gt ge`	`t = x relop y`	Comparación con resultado booleano.

Tabla 8: Instrucciones de datos y expresiones.

Control de flujo.

op	Forma textual	Semántica
`label`	`L:`	Define un destino simbólico.
`goto`	`goto L`	Salto incondicional.
`if_true`	`if x goto L`	Salta cuando `x` es verdadero.
`if_false`	`ifFalse x goto L`	Salta cuando `x` es falso.

Tabla 9: Instrucciones de control de flujo.

Funciones, procedimientos y closures.

op	Forma textual	Semántica
`begin_func`, `end_func`	`func f:`, `endfunc f`	Delimita una rutina.
`param_in`	`param_in x, n`	Vincula el parámetro formal número `n`.
`param`	`param x`	Pasa un argumento ya evaluado.
`call`	`t = call f, n`	Invoca una función con `n` argumentos.
`call_method`	`t = call_method C.m, n`	Invoca un método con receptor implícito.
`return`	`return x`	Devuelve control y un valor opcional.
`capture`	`capture x from access_link`	Declara acceso a un nombre no local.
`static_link`	`static_link f = current_env`	Prepara el entorno léxico de una llamada a función anidada.

Tabla 10: Instrucciones de funciones y closures.

Clases, objetos y listas.

op	Forma textual	Semántica
`begin_class`, `end_class`	`class C:`, `endclass C`	Delimita metadatos y métodos de clase.
`field_decl`, `field_init`	`field x : T`, `field x = v`	Define un atributo y su valor inicial.
`new_object`	`t = new C`	Reserva una instancia.
`load_field`	`t = obj.campo`	Lee un atributo.
`store_field`	`obj.campo = v`	Escribe un atributo.
`new_list`	`t = newlist n`	Crea una lista con `n` posiciones.
`load_index`	`t = lista[i]`	Lee un elemento.
`store_index`	`lista[i] = v`	Escribe un elemento.
`length`	`t = length lista`	Límite usado por `foreach`.

Tabla 11: Instrucciones de clases, objetos y listas.

Otras operaciones. `print`, `try_begin`, `catch` y `try_end` representan las construcciones que no requieren descomposición adicional en esta fase. `begin_program` y `end_program` delimitan el código de nivel global.

7. Traducción de Compiscript a TAC

Esta sección muestra, para cada familia de construcciones, el fragmento fuente y el TAC que genera el compilador. **Todos los listados son la salida real de `compiscript_cli`**; por brevedad se omiten las líneas `begin_program` y `end_program` y las declaraciones previas que no son relevantes.

7.1. Expresiones aritméticas, precedencia y asociatividad

Cada operador binario consume dos direcciones y produce un temporal. El orden de emisión respeta la precedencia: la multiplicación se emite antes que la suma porque el árbol la coloca más profunda.

```
let a = 3;
let b = 5;
let x = a + b * 4;
```

```
declare @x#2 : integer
%t0 = @b#1 * 4
%t1 = @a#0 + %t0
@x#2 = %t1
```

Los operadores de igual precedencia se asocian a la izquierda y los argumentos se evalúan de izquierda a derecha. Las constantes no se pliegan: `1 + 2` genera una suma explícita, porque la propagación de constantes pertenece a una fase de optimización posterior.

7.2. Expresiones lógicas y comparaciones

Las seis comparaciones producen un temporal booleano (`==`, `!=`, `<`, `<=`, `>`, `>=`). `&&` y `||` se traducen con **saltos de cortocircuito**: el operando derecho no se evalúa si el izquierdo ya determina el resultado.

```
let a: boolean = true;
let b: boolean = false;
let c: boolean =
  a && b || !a;
```

```
ifFalse @a#0 goto L_and_false_2
ifFalse @b#1 goto L_and_false_2
%t1 = true
goto L_and_end_3
L_and_false_2:
%t1 = false
L_and_end_3:
if %t1 goto L_or_true_0
%t1 = !@a#0
if %t1 goto L_or_true_0
%t0 = false
goto L_or_end_1
L_or_true_0:
%t0 = true
L_or_end_1:
@c#2 = %t0
```

7.3. Condicionales

`if/else` evalúa la condición, salta a la rama `else` cuando es falsa y, al terminar la rama `then`, salta al final. Sin `else` el salto falso va directamente a la etiqueta final. El operador ternario usa el mismo esquema y deja el valor en un temporal.

```
if (nota >= 61) {
  estado = "aprobado";
} else {
  estado = "reprobado";
}
```

```
%t0 = @nota#0 >= 61
ifFalse %t0 goto L_if_else_0
@estado#1 = "aprobado"
goto L_if_end_1
L_if_else_0:
@estado#1 = "reprobado"
L_if_end_1:
```

7.4. Ciclos

`while` coloca la prueba al inicio; `do-while` coloca la prueba al final, de modo que el cuerpo se ejecuta al menos una vez; `for` agrega una etiqueta de incremento que es el destino de `continue`.

```
let x: integer = 0;
while (x < 10) {
  x = x + 1;
}
```

```
L_while_cond_0:
%t0 = @x#0 < 10
ifFalse %t0 goto L_while_end_1
%t0 = @x#0 + 1
@x#0 = %t0
goto L_while_cond_0
L_while_end_1:
```

```
do {
  n = n - 1;
} while (n > 0);
```

```
L_do_body_0:
%t0 = @n#0 - 1
@n#0 = %t0
L_do_cond_1:
%t0 = @n#0 > 0
if %t0 goto L_do_body_0
L_do_end_2:
```

7.4.1. break y continue

`break` salta a la etiqueta de salida del ciclo que lo contiene y `continue` salta a la etiqueta de condición (en `while` y `do-while`) o de incremento (en `for` y `foreach`). El generador mantiene una pila de destinos para cada ciclo, así que los ciclos anidados resuelven siempre el más interno.

```
for (let i: integer = 0;
     i < 4; i = i + 1) {
  if (i == 1) { continue; }
  if (i == 3) { break; }
  print(i);
}
```

```
L_for_cond_0:
%t0 = @i#0 < 4
ifFalse %t0 goto L_for_end_2
%t0 = @i#0 == 1
ifFalse %t0 goto L_if_end_4
goto L_for_inc_1        // continue
L_if_end_4:
%t0 = @i#0 == 3
ifFalse %t0 goto L_if_end_6
goto L_for_end_2        // break
L_if_end_6:
print @i#0
L_for_inc_1:
%t0 = @i#0 + 1
@i#0 = %t0
goto L_for_cond_0
L_for_end_2:
```

(Los comentarios `// continue` y `// break` se añadieron al listado para facilitar la lectura; el generador no los emite.)

7.5. switch

Cada `case` se compara con el selector mediante `eq`; si la comparación falla se salta al siguiente caso. Al terminar el cuerpo de un caso se salta al final: **no hay caída implícita**, coherente con la regla del proyecto que reserva `break` para los ciclos. El `default` es el último destino.

```
switch (op) {
  case true:  print(1);
  case false: print(0);
  default:    print(2);
}
```

```
%t0 = @op#0 == true
ifFalse %t0 goto L_case_next_1
print 1
goto L_switch_end_0
L_case_next_1:
%t0 = @op#0 == false
ifFalse %t0 goto L_case_next_2
print 0
goto L_switch_end_0
L_case_next_2:
print 2
L_switch_end_0:
```

7.6. Funciones, parámetros, retorno y recursividad

Una función se delimita con `func`/`endfunc`. Sus parámetros se vinculan con `param_in` y la llamada evalúa los argumentos de izquierda a derecha, emite un `param` por cada uno y luego un `call` con la cantidad de argumentos. Las funciones pueden llamarse antes de aparecer en el texto porque el análisis semántico predeclara sus firmas y el TAC usa etiquetas simbólicas; la recursividad usa la misma etiqueta de función.

```
function factorial(n: integer):
    integer {
  if (n <= 1) return 1;
  return n * factorial(n - 1);
}
let r = factorial(5);
```

```
func fn_factorial_0:
param_in %n#1, 0
%t0 = %n#1 <= 1
ifFalse %t0 goto L_if_end_1
return 1
L_if_end_1:
%t0 = %n#1 - 1
param %t0
%t1 = call fn_factorial_0, 1
%t0 = %n#1 * %t1
return %t0
endfunc fn_factorial_0
declare @r#2 : integer
param 5
%t0 = call fn_factorial_0, 1
@r#2 = %t0
```

Obsérvese cómo `%t0` se reutiliza tras la llamada recursiva: el valor de `n - 1` ya fue consumido por `param`, de modo que el temporal vuelve al pool y `n * factorial(n - 1)` lo toma de nuevo.

7.7. Funciones anidadas y closures

Cuando una función anidada resuelve un nombre declarado fuera de ella, la semántica registra la captura (`captures` / `captured_by`). El generador la convierte en tres elementos: una instrucción `capture` al inicio de la función, accesos `env[...]` al nombre capturado y una instrucción `static_link` antes de cada llamada, que prepara el enlace de acceso con el entorno de definición.

```
function crearAcumulador(base: integer):
    integer {
  function sumar(valor: integer): integer {
    return base + valor;
  }
  return sumar(10);
}
let resultado: integer = crearAcumulador(5)
    ;
print(resultado);
```

```
func fn_crearAcumulador_0:
param_in %base#1, 0
func fn_sumar_2:
param_in %valor#3, 0
capture env[%base#1] from access_link
%t0 = env[%base#1] + %valor#3
return %t0
endfunc fn_sumar_2
param 10
static_link fn_sumar_2 = current_env
%t0 = call fn_sumar_2, 1
return %t0
endfunc fn_crearAcumulador_0
declare @resultado#4 : integer
param 5
%t0 = call fn_crearAcumulador_0, 1
@resultado#4 = %t0
print @resultado#4
```

7.8. Clases, objetos y métodos

`new C(args)` reserva la instancia con `new_object` y llama al constructor con `call_method`. En un método, el objeto implícito es `%this`: se vincula como primer `param_in` con la marca `receiver` y no cuenta en la aridad declarada por el usuario. Las lecturas y escrituras de atributos usan `load_field` y `store_field`, y la invocación de un método pasa el receptor como primer `param`. Los atributos heredados se ubican antes que los de la clase derivada (sección 9). El ejemplo completo está en la sección 7.11.

7.9. Listas y estructuras de datos

Una lista es una referencia dinámica. Un literal se traduce a `newlist` más un `store_index` por elemento; el acceso con índice usa `load_index`/`store_index`. `foreach` se baja a un índice auxiliar, `length`, una comparación, la lectura del elemento y un salto (ver el ejemplo integral).

```
let datos = [10, 20];
datos[1] = datos[0] + 5;
```

```
%t0 = newlist 2
%t0[0] = 10
%t0[1] = 20
@datos#0 = %t0
%t0 = @datos#0[1]
%t1 = @datos#0[0]
%t2 = %t1 + 5
@datos#0[1] = %t2
```

La primera lectura (`%t0 = @datos#0[1]`) corresponde al lado izquierdo de la asignación: el generador conserva la localización asignable evaluando primero el destino. Es una carga redundante cuyo valor no se usa; una fase de optimización posterior puede eliminarla.

7.10. try/catch

El subconjunto de Compiscript no define todavía un ABI de excepciones, por lo que `try/catch` se conserva mediante operaciones abstractas.

```
try {
  print(1);
} catch (e) {
  print(2);
}
```

```
try_begin L_catch_0
print 1
goto L_try_end_1
L_catch_0:
catch @e#0
print 2
L_try_end_1:
try_end
```

7.11. Ejemplo integral

El programa `codigo_intermedio.cps` reúne clase, constructor, método, atributos, lista, `foreach`, función, llamadas y `print`. Es el ejemplo que se usa en las capturas del IDE.

```
1  class Acumulador {
2    let total: integer;
3
4    function constructor(inicial: integer) {
5      this.total = inicial;
6    }
7
8    function sumar(valor: integer): integer {
9      this.total = this.total + valor;
10     return this.total;
11   }
12 }
13
14 function procesar(datos: integer[]): integer {
15   let acumulador = new Acumulador(0);
16   foreach (dato in datos) {
17     acumulador.sumar(dato);
18   }
19   return acumulador.total;
20 }
21
22 let valores = [1, 2, 3, 4];
23 let resultado = procesar(valores);
24 print(resultado);
```

Listing 1: `codigo_intermedio.cps`: clase, método, lista y `foreach`.

La traducción produce **61 instrucciones** (numeradas desde 0, igual que el campo `index` del cuádruplo):

```
0  begin_program
1  class Acumulador:
2  field total : integer
3  func fn_Acumulador_constructor_3:
4  param_in %this, receiver
5  param_in %inicial#5, 0
6  %t0 = %this.total
7  %this.total = %inicial#5
8  return
9  endfunc fn_Acumulador_constructor_3
10 func fn_Acumulador_sumar_4:
11 param_in %this, receiver
12 param_in %valor#6, 0
13 %t0 = %this.total
14 %t1 = %this.total
15 %t2 = %t1 + %valor#6
```

```
%this.total = %t2
%t2 = %this.total
return %t2
endfunc fn_Acumulador_sumar_4
endclass Acumulador
func fn_procesar_1:
param_in %datos#7, 0
declare %acumulador#8 : Acumulador
%t0 = new Acumulador
param %t0
param 0
call_method fn_Acumulador_constructor_3, 1
%acumulador#8 = %t0
declare %dato#9 : integer
%t0 = 0
%t1 = length %datos#7
L_foreach_cond_0:
%t2 = %t0 < %t1
ifFalse %t2 goto L_foreach_end_2
%t2 = %datos#7[%t0]
%dato#9 = %t2
param %acumulador#8
param %dato#9
%t2 = call_method fn_Acumulador_sumar_4, 1
L_foreach_inc_1:
%t2 = %t0 + 1
%t0 = %t2
goto L_foreach_cond_0
L_foreach_end_2:
%t1 = %acumulador#8.total
return %t1
endfunc fn_procesar_1
declare @valores#10 : integer[]
%t0 = newlist 4
%t0[0] = 1
%t0[1] = 2
%t0[2] = 3
%t0[3] = 4
@valores#10 = %t0
declare @resultado#11 : integer
param @valores#10
%t0 = call fn_procesar_1, 1
@resultado#11 = %t0
print @resultado#11
end_program
```

Listing 2: TAC generado para `codigo_intermedio.cps`.

Puntos que conviene notar al leerlo:

- El constructor y el método reciben `%this` como primer `param_in`; las variables locales (`%`) y las globales (`@`) se distinguen por el prefijo.
- `new Acumulador(0)` produce un `new`, dos `param` (receptor y argumento) y un `call_method` con *un* solo argumento explícito: el receptor no cuenta.
- `foreach` usa tres temporales a la vez (`%t0` como índice, `%t1` como límite y `%t2` para la prueba y el elemento); ese es el máximo simultáneo del programa.
- El resultado de `acumulador.sumar(dato)` no se usa y su temporal se reutiliza de inmediato en el incremento.

8. Manejo de temporales

Cada función y el nivel global tienen un `TempPool` independiente. El algoritmo es el siguiente:

1. `newTemp` toma un nombre de la lista libre del pool o, si está vacía, crea el siguiente `%tN`.
2. Una instrucción se emite **antes** de liberar sus operandos.
3. Cuando el valor de un temporal ya fue consumido, `releaseTemp` lo devuelve a la lista libre.
4. El pool lleva tres contadores: temporales creados, reutilizaciones y máximo de valores simultáneamente vivos.
5. El máximo simultáneo, multiplicado por 8 bytes, determina el espacio de temporales del registro de activación; no depende del número total de expresiones del programa.

```
std::string TacGenerator::newTemp() {
  TempPool &pool = temp_pools_.back();
  std::string name;
  if (!pool.free.empty()) {            // reutiliza un temporal liberado
    name = pool.free.back();
    pool.free.pop_back();
    ++pool.reused;
  } else {                              // crea uno nuevo
    name = "%t" + std::to_string(pool.next_id++);
    ++pool.created;
  }
  pool.live.insert(name);
  pool.max_live = std::max(pool.max_live, pool.live.size());
  return name;
}

void TacGenerator::releaseTemp(const std::string &name) {
  if (pool.live.erase(name))
    pool.free.push_back(name);          // vuelve a la lista libre
}
```

Listing 3: Asignación y reciclaje de temporales (`tac_generator.cpp`, simplificado).

El efecto se aprecia en un programa que suma tres subtotales: pese a tener cinco expresiones intermedias, solo se crean dos temporales.

```
let a = 1 + 2;
let b = 3 + 4;
let c = 5 + 6;
print(a + b + c);
```

```
%t0 = 1 + 2
@a#0 = %t0
%t0 = 3 + 4
@b#1 = %t0
%t0 = 5 + 6
@c#2 = %t0
%t0 = @a#0 + @b#1
%t1 = %t0 + @c#2
print %t1
```

Programa	Creados	Reutilizaciones	Máx. vivos
`a + b * 4`	2	0	2
`while (x < 10)`	1	1	1
`factorial` (recursivo)	3	2	2
Suma de tres subtotales	2	3	2
`for` con `continue`/`break`	1	3	1
`closures.cps`	3	0	1
`codigo_intermedio.cps`	8	7	3

Tabla 12: Métricas de temporales reportadas por el generador (campos `temporaries_created`, `temporaries_reused` y `max_live_temporaries`).

No se realiza propagación de constantes, eliminación de código muerto TAC ni asignación de registros físicos: esas optimizaciones pertenecen a fases posteriores.

9. Tabla de símbolos y entornos de ejecución

9.1. Información agregada para la generación de código

La primera acción del `TacGenerator` (`layoutSymbols`, ejecutada en su constructor) completa la tabla que dejó el análisis semántico. Cada `Symbol` recibe cinco campos nuevos y cada `Scope` dos:

Estructura	Campo	Significado
`Symbol`	`size`	Tamaño del valor en bytes.
	`offset`	Desplazamiento en globales, objetos o frames (`-1` si no aplica).
	`lexical_depth`	Profundidad del entorno que lo declara.
	`storage`	`global`, `local`, `captured-local`, `parameter`, `field`, `code` o `type`.
	`label`	Etiqueta de funciones, métodos y clases.
`Scope`	`lexical_depth`	Número de ancestros hasta la raíz.
	`frame_size`	Tamaño del registro de activación (entornos de función).

Tabla 13: Campos que la fase de código intermedio agrega a la tabla de símbolos.

9.2. Supuestos de tamaño y alineación

- `boolean`: 1 byte; `integer`: 4 bytes; `float`: 8 bytes.
- `string`, listas, objetos y funciones: referencia de 8 bytes.
- Cada *slot* del layout se alinea a 8 bytes para mantener los desplazamientos simples.
- Los atributos heredados preceden a los de la clase derivada.

El programa siguiente permite verificar el layout. Los datos de la tabla son la salida real del compilador.

```
class A { let x: integer; let y: float; }
class B : A { let z: boolean; }
function f(a: integer, b: float): integer {
  let t: boolean = true;
  let n: integer = a;
  return n;
}
let o: B = new B();
let g: integer = f(1, 2.5);
```

Listing 4: Programa usado para ilustrar offsets, tamaños y herencia.

Símbolo	Clase	Almacenamiento	Offset	Tamaño
`A`, `B`	clase	`type`	—	8 B
`f`	función	`code`	—	8 B
`x` (en `A`)	atributo	`field`	0	4 B
`y` (en `A`)	atributo	`field`	8	8 B
`z` (en `B`)	atributo	`field`	16	1 B
`a` (en `f`)	parámetro	`parameter`	32	4 B
`b` (en `f`)	parámetro	`parameter`	40	8 B
`t` (en `f`)	local	`local`	−8	1 B
`n` (en `f`)	local	`local`	−16	4 B
`o`	global	`global`	0	8 B
`g`	global	`global`	8	4 B

Tabla 14: Tabla de símbolos extendida del programa anterior.

`z` comienza en el desplazamiento 16 porque `B` hereda los 16 bytes que ocupan `x` e `y`. Los parámetros se numeran hacia arriba desde 32 (después de los cuatro campos fijos del frame) y las locales hacia abajo desde −8.

9.3. Registros de activación

El frame abstracto usa desplazamientos relativos al *frame pointer*, siguiendo el modelo del libro del dragón para la pila de control:

Offset	Contenido	Campo de `ActivationRecord`
0	Enlace de control (frame del llamador)	`control_link_offset`
8	Enlace de acceso (entorno léxico para closures)	`access_link_offset`
16	Dirección de retorno	`return_address_offset`
24	Valor de retorno	`return_value_offset`
32 . . .	Parámetros formales	`parameter_bytes`
Negativos	Variables locales y capturadas	`local_bytes`
Área adicional	Máximo de temporales simultáneamente vivos	`temporary_bytes`

Tabla 15: Diseño del registro de activación.

El tamaño total es

$$\text{frame} = 32 + \text{parámetros} + \text{locales} + 8 \cdot \text{máx. temporales vivos}.$$

Para `codigo_intermedio.cps` el generador reporta:

Función	Etiqueta	Par.	Loc.	Temp.	Frame
`procesar`	`fn_procesar_1`	8 B	16 B	24 B	80 B
`Acumulador.constructor`	`fn_Acumulador_constructor_3`	8 B	0 B	8 B	48 B
`Acumulador.sumar`	`fn_Acumulador_sumar_4`	8 B	0 B	24 B	64 B

Tabla 16: Registros de activación de `codigo_intermedio.cps`.

Por ejemplo, `procesar` tiene $32 + 8 + 16 + 24 = 80$ bytes: un parámetro (`datos`), dos locales (`acumulador` y `dato`) y tres temporales vivos como máximo.

9.4. Acceso a datos no locales

Las funciones anidadas acceden a las variables de su entorno de definición por el **enlace de acceso** (offset 8). La tabla registra esa relación en ambos sentidos: la función guarda en `captures` los símbolos que necesita y cada símbolo capturado se marca con `captured_by`; el almacenamiento de un local capturado pasa a `captured-local`. En el TAC esto se materializa con `capture`, con los accesos `env[...]` y con `static_link` en cada llamada. El registro de activación de `sumar` (en `closures.cps`) lista, en la columna de capturas, el símbolo `#1` (`base`).

9.5. Interacción con las fases del compilador

La tabla de símbolos participa en toda la cadena: el análisis semántico la construye y registra los *bindings*; el generador de código la consulta para resolver cada identificador, la **completa** con tamaños, offsets, etiquetas y frames y vuelve a escribir en ella el tamaño final de cada frame cuando termina de contar los temporales; y el serializador la envía íntegra al IDE, donde se puede revisar por entorno y por ámbito.

10. El IDE

El IDE es la misma aplicación web servida por Flask de la fase anterior, con una pestaña, una tarjeta de resumen y varias columnas nuevas. Permite escribir, cargar, guardar y descargar archivos `.cps`, compilar con `Ctrl/Cmd + Enter` o con el botón principal, y revisar los resultados de cada fase. Las capturas siguientes se tomaron ejecutando el compilador real sobre los ejemplos incluidos en el repositorio.

10.1. Pestaña Código TAC

La pestaña muestra primero las métricas del generador (instrucciones, temporales creados, reutilizaciones y máximo simultáneo) y la representación textual, con un botón para copiarla.

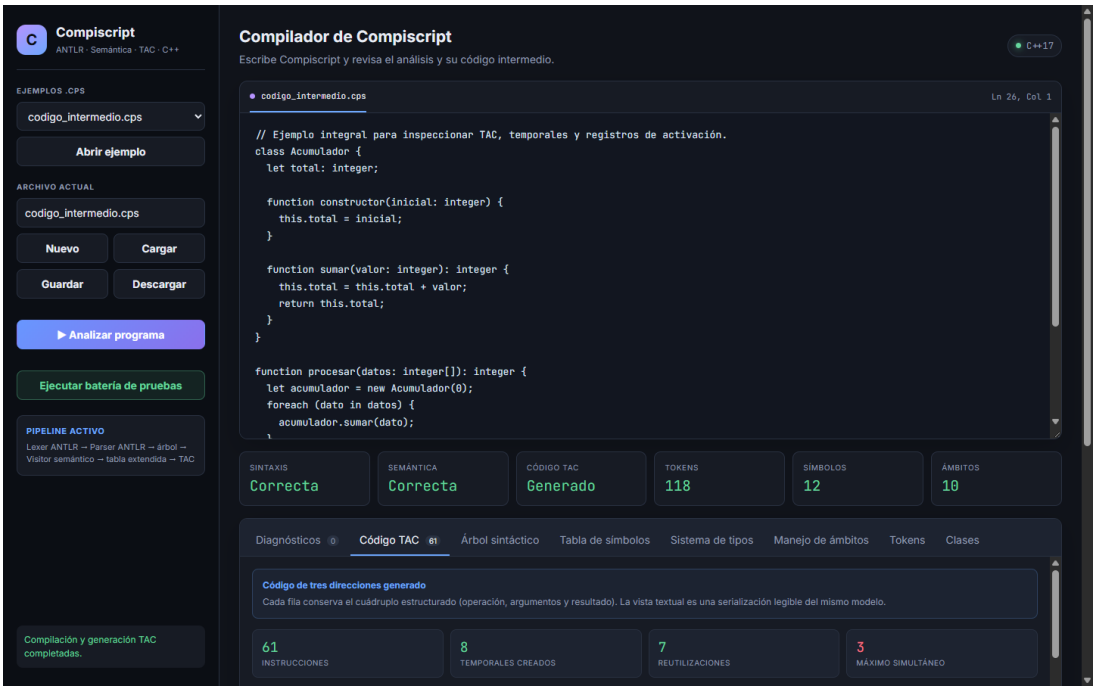


Figura 1: Compilación de `codigo_intermedio.cps`: sintaxis y semántica correctas, *Código TAC: Generado*, 118 tokens, 12 símbolos y 10 ámbitos.

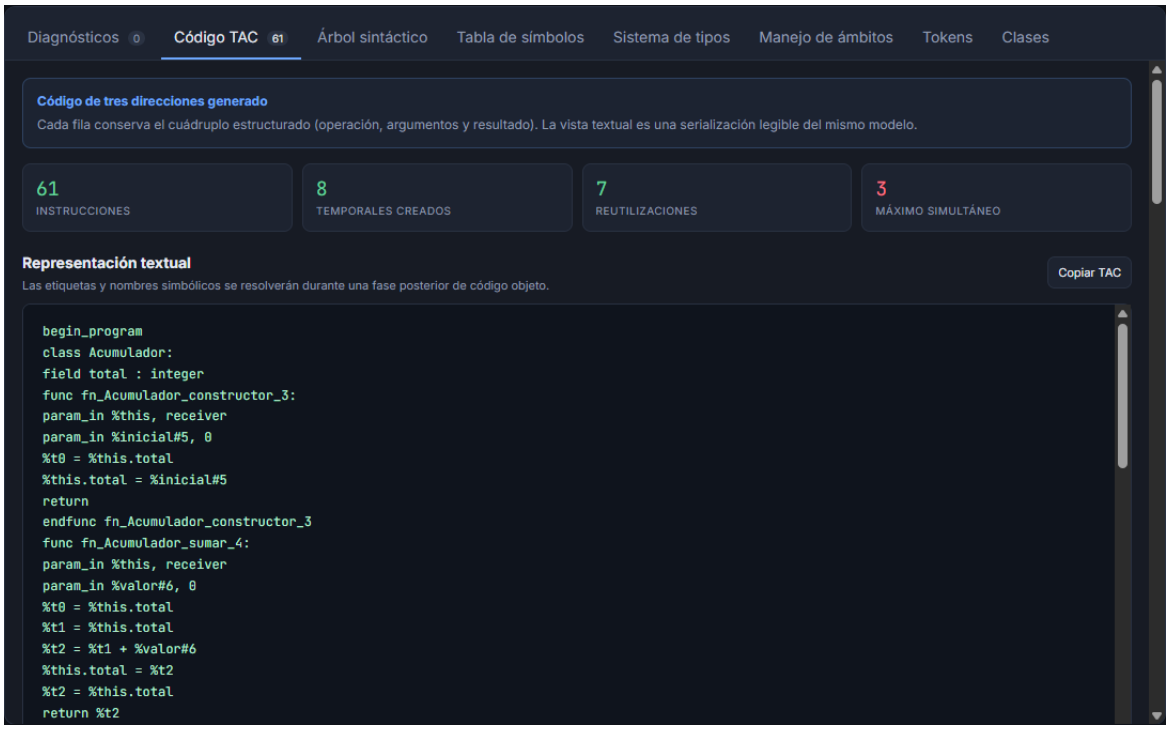


Figura 2: Métricas y representación textual del TAC de `codigo_intermedio.cps`.

Debajo de la vista textual se presenta la tabla de cuádruplos, con la operación, el resultado, los dos argumentos y la posición de origen (`línea:columna`) en el archivo fuente. Un guion indica que la forma de instrucción no usa ese operando.

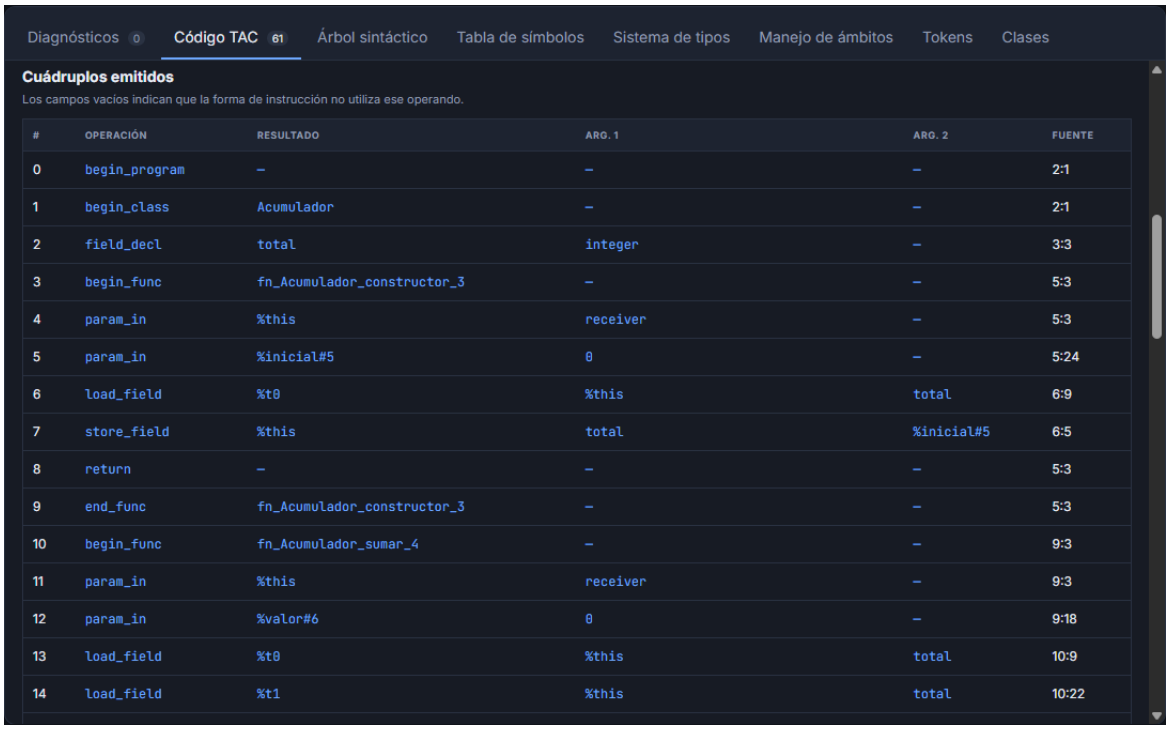


Figura 3: Tabla de cuádruplos emitidos.

Al final se listan los registros de activación con su etiqueta, profundidad léxica, bytes de parámetros, locales y temporales, tamaño total y capturas.

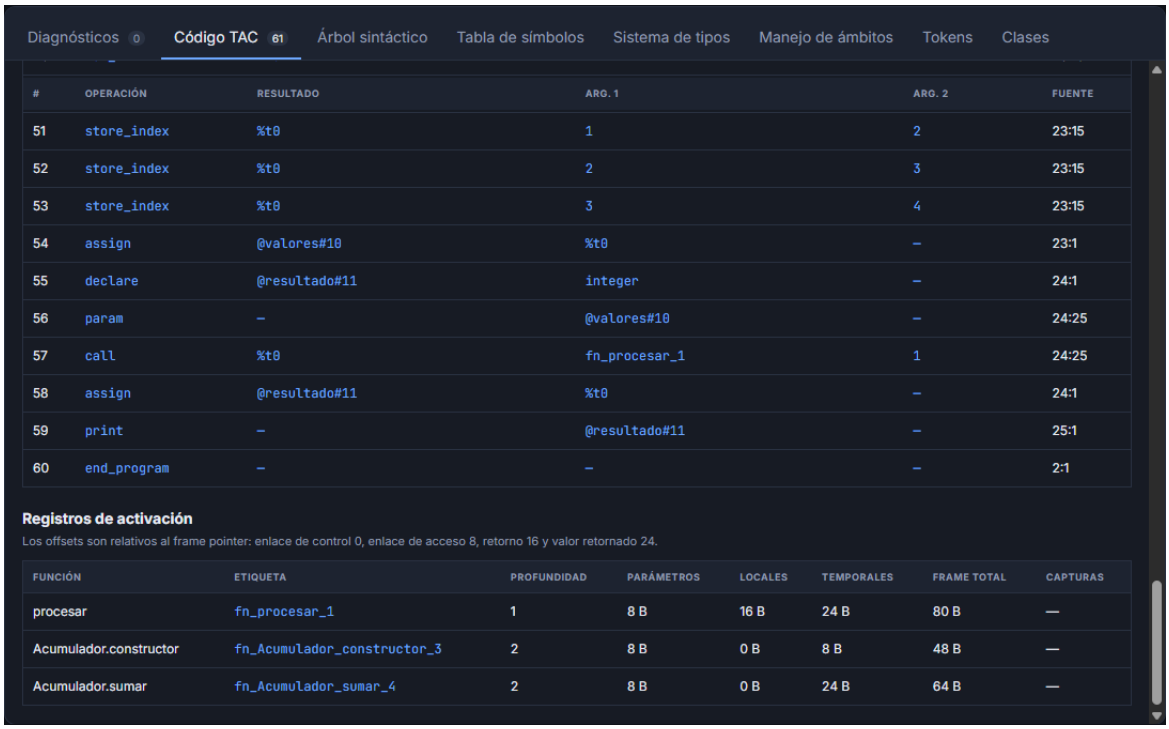


Figura 4: Últimos cuádruplos y registros de activación de cada función y método.

10.2. Closures en el IDE

Para `closures.cps` el TAC incluye `capture` y `static_link`, y el frame de `sumar` registra la captura de `base`.

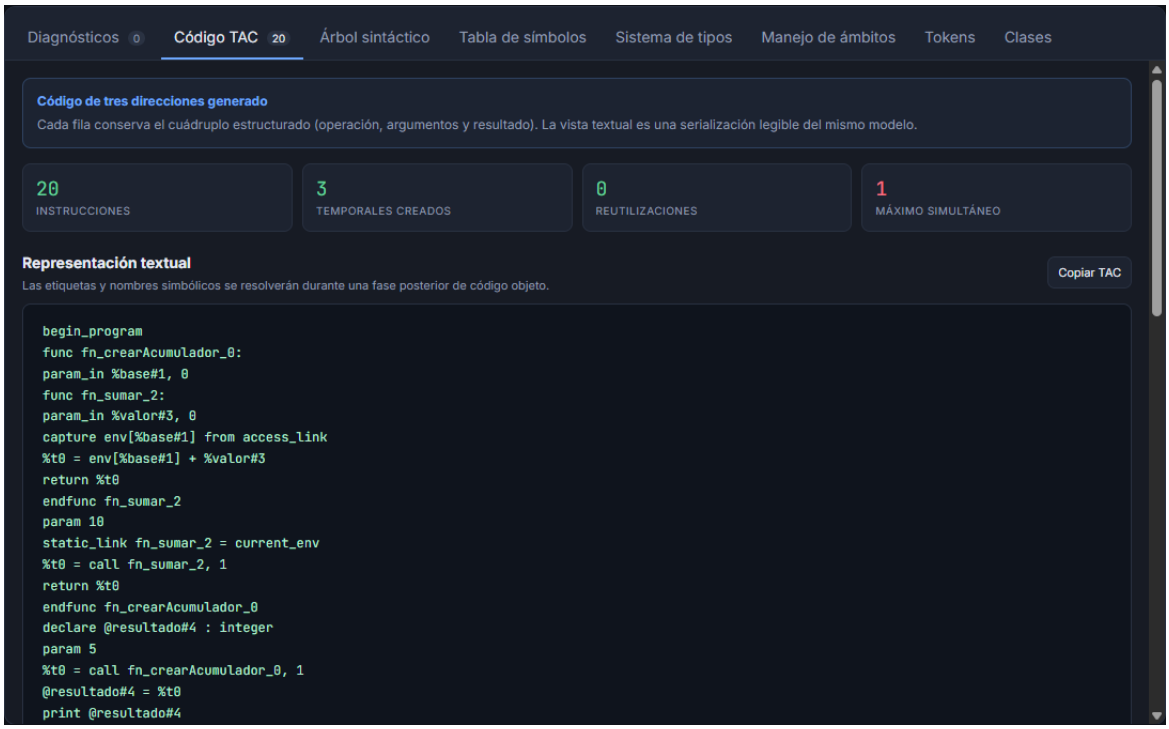


Figura 5: TAC de `closures.cps`: `capture`, acceso `env[...]` y `static_link`.

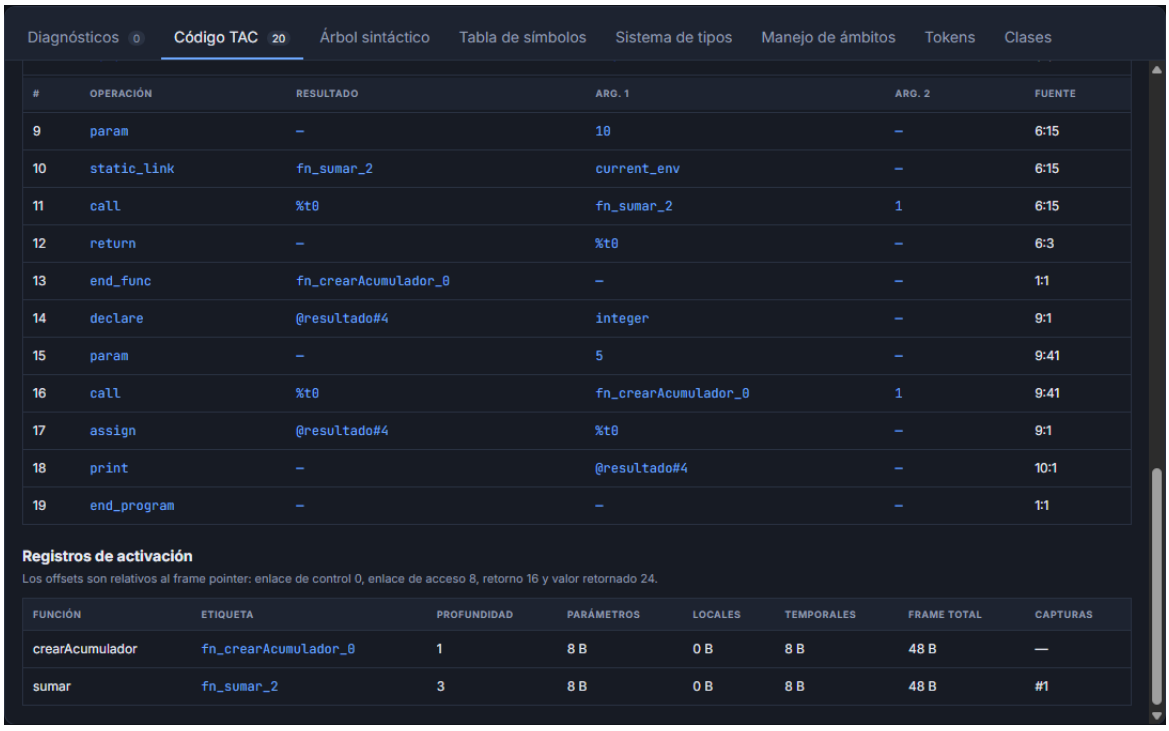


Figura 6: Frames de `closures.cps`: `sumar` (profundidad 3) captura el símbolo #1.

10.3. Tabla de símbolos y ámbitos extendidos

La tabla de símbolos ahora incluye las columnas *Almacenamiento* y *Offset / tamaño*, y la vista de ámbitos agrega *Profundidad* y *Frame*.

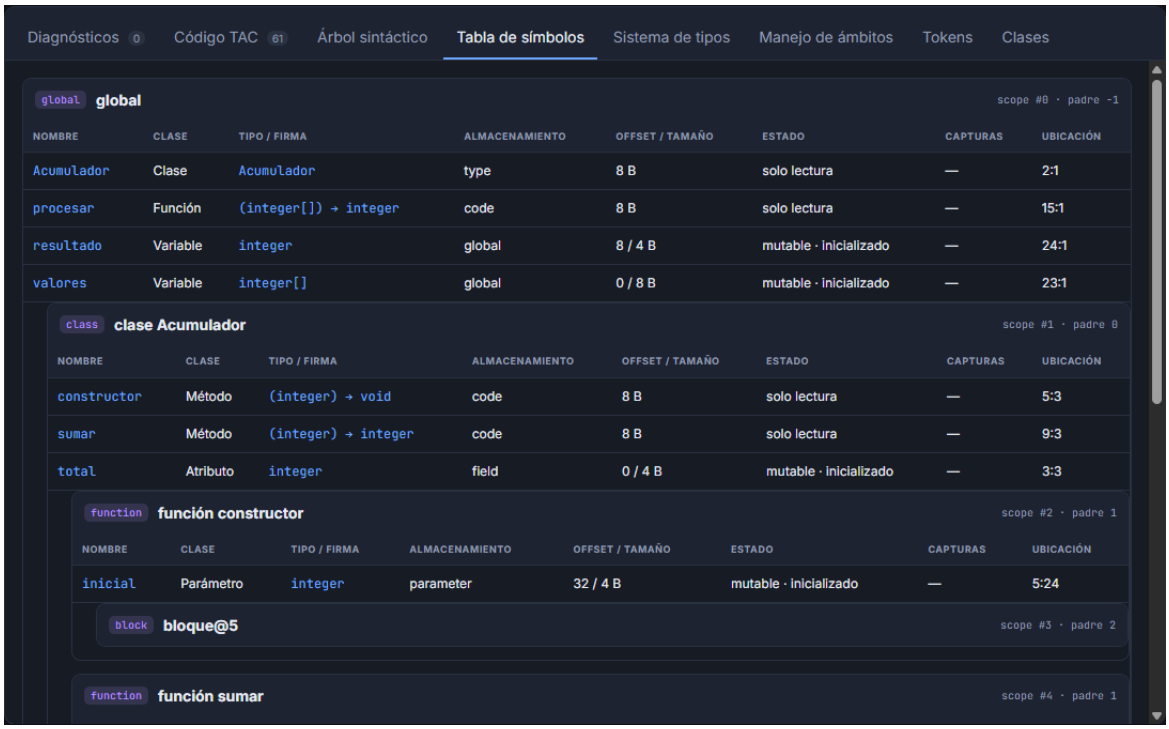


Figura 7: Tabla de símbolos con almacenamiento, offset y tamaño. `total` es un atributo en el offset 0; `inicial` es un parámetro en el offset 32.

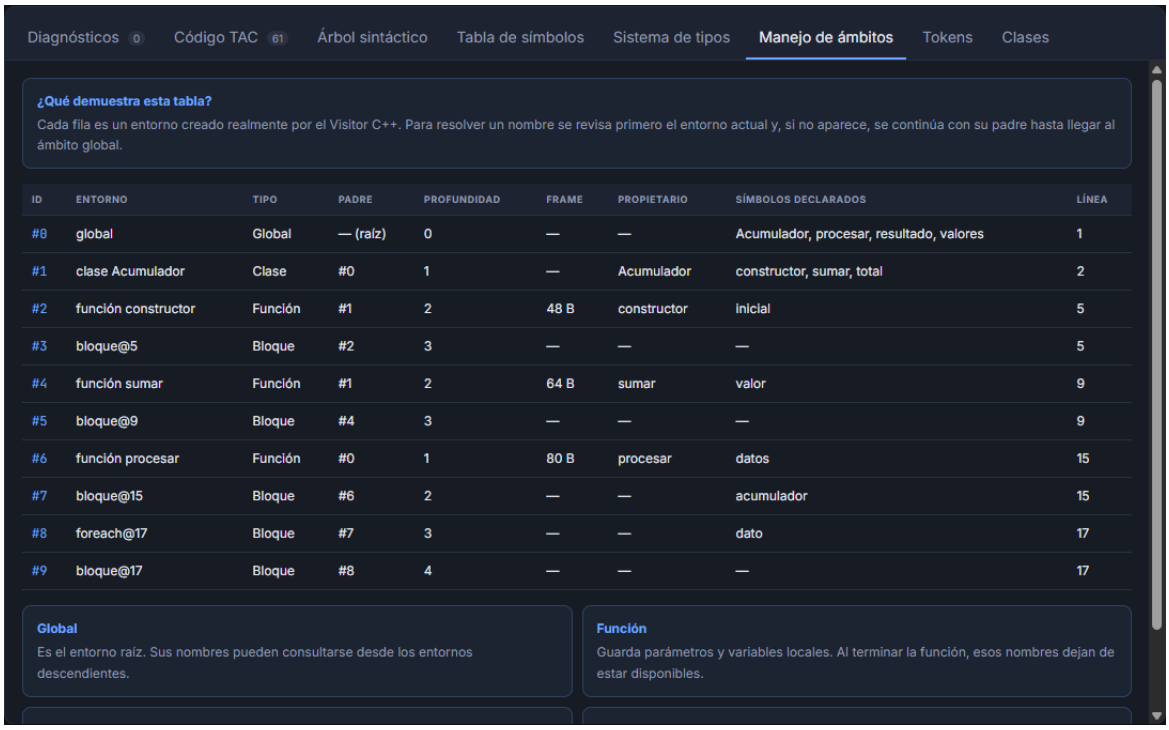


Figura 8: Vista de ámbitos con profundidad léxica y tamaño de frame: 48 B para el constructor, 64 B para `sumar` y 80 B para `procesar`.

10.4. Reporte de errores

Cuando el programa tiene errores, el IDE sigue mostrando los diagnósticos con su código, categoría y ubicación, y la tarjeta *Código TAC* indica *Omitido*. En `errores_semanticos.cps` la sintaxis es correcta, la semántica reporta 17 errores y no se genera código:

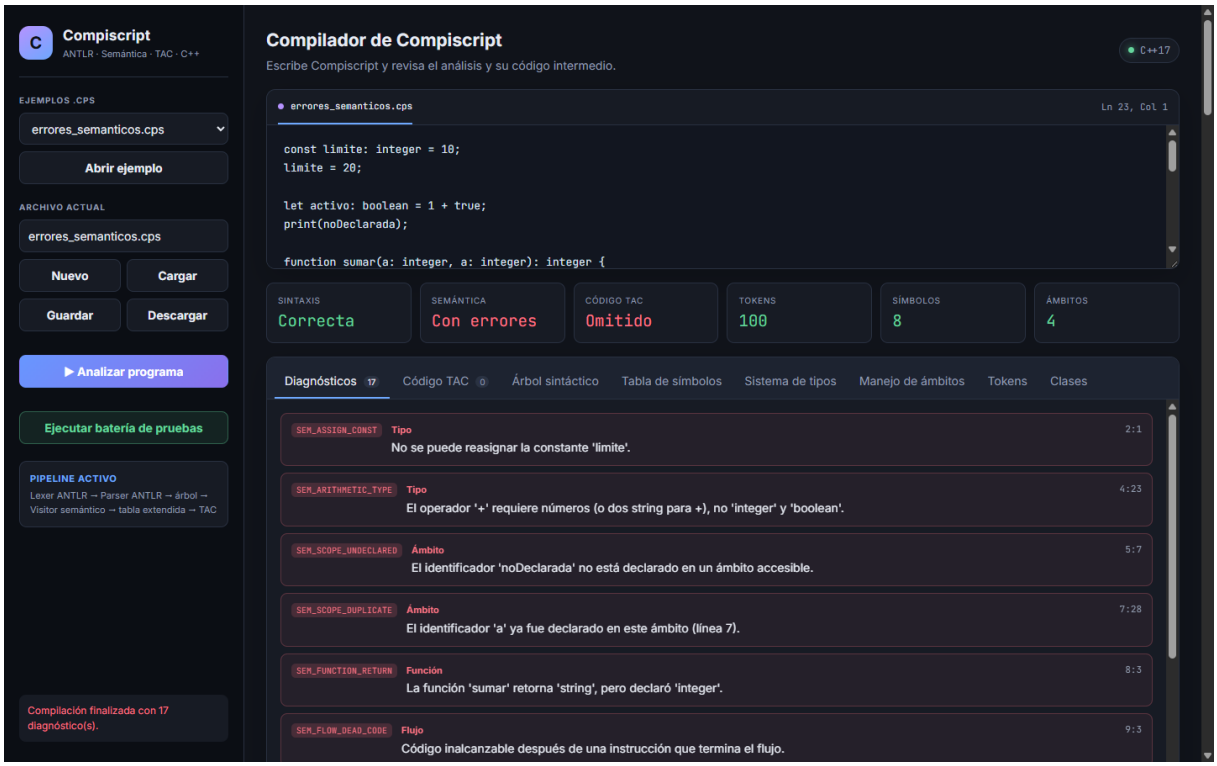


Figura 9: Compilación de `errores_semanticos.cps`: 17 diagnósticos semánticos y *Código TAC: Omitido*.

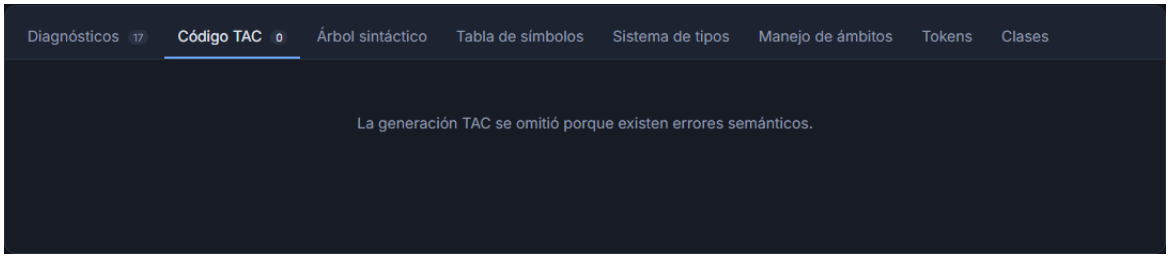


Figura 10: La pestaña *Código TAC* explica por qué la generación se omitió.

Ante un error sintáctico, tanto la semántica como el TAC se omiten:

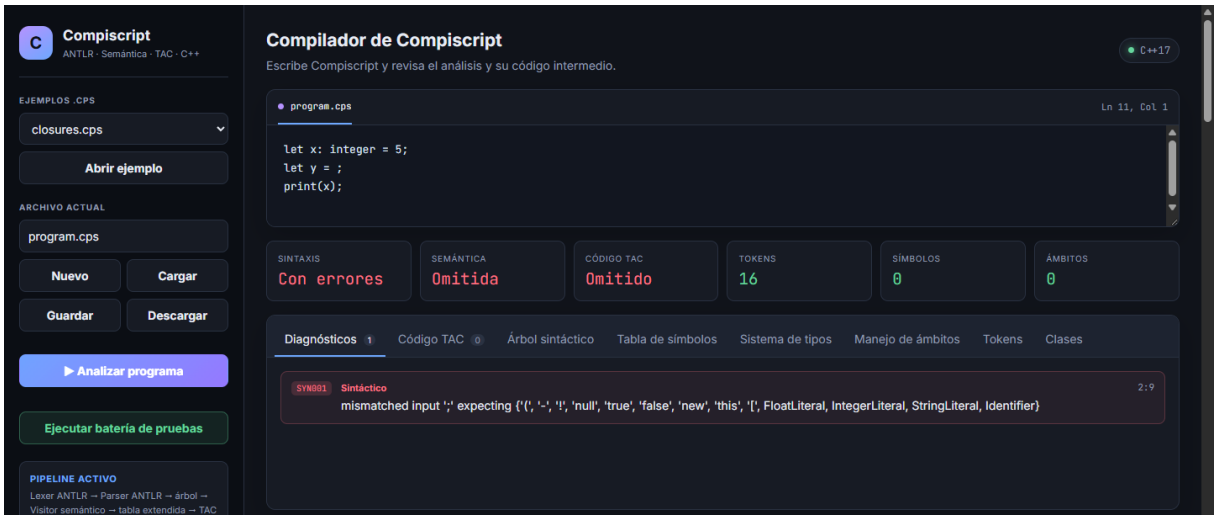


Figura 11: Entrada con un error sintáctico (`let y = ;`): sintaxis con errores, semántica y TAC omitidos.

11. Batería de pruebas

La batería está en `backend/tests/compiscript_tests.cpp` y contiene ahora **107 escenarios**: 51 programas válidos y 56 errores esperados. Sigue sin depender de ningún *framework* externo y enlaza directamente contra `compiscript_core`, de modo que prueba el mismo servicio que usan el CLI y el IDE.

El criterio de un caso negativo sigue siendo estricto: no basta con que el programa falle, debe fallar **con el código de diagnóstico correcto**, y en los casos de TAC además debe quedar marcado como omitido y sin instrucciones.

Categoría	Aprobadas	Total
Proceso completo	4	4
Sistema de tipos	20	20
Manejo de ámbito	7	7
Funciones y procedimientos	13	13
Control de flujo	20	20
Clases y objetos	13	13
Listas y estructuras de datos	11	11
Reglas generales	7	7
Generación de código intermedio (nueva)	**12**	**12**
Total	**107**	**107**

Tabla 17: Resultado de la batería por categoría.

11.1. Escenarios de generación de código

Tema	Entrada	Qué verifica
Precedencia	`2 + 3 * 4`	`mul` se emite antes que `add` y no hay diagnósticos.
Cortocircuito	`(true && false) || true`	Hay `if_true`, `if_false` y etiquetas.
Control de flujo	`for` con `continue` y `break`	Al menos 3 `goto` y 5 etiquetas.
`switch`	selector booleano con dos `case`	Exactamente dos comparaciones `eq`; sin caída.
Funciones	`factorial` recursivo	`begin_func`, `param_in`, `call`, `return` y frame no vacío.
Closures	función anidada que usa `base`	Frame con símbolos capturados, `capture` y `static_link`.
Clases	constructor, campo y método	`new_object`, `load_field`, `store_field` y `call_method`.
Listas	creación, lectura y escritura	`new_list`, `load_index` y `store_index`.
Temporales	tres sumas encadenadas	Se reutiliza al menos un temporal ya consumido.
Tabla extendida	globales, parámetros y locales	Todo símbolo con valor tiene `storage` y `offset`; el frame trae parámetros y locales y mide al menos 32 B.
Omisión semántica	`let x: integer = true;`	`SEM_ASSIGN_TYPE`, TAC omitido y sin instrucciones.
Omisión sintáctica	`let x = ;`	Semántica y TAC omitidos y sin instrucciones.

Tabla 18: Los 12 escenarios agregados en el commit `66f83c3`.

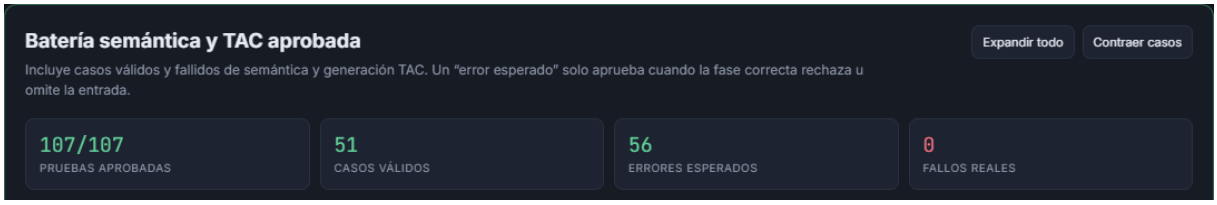


Figura 12: Batería ejecutada desde el IDE: 107/107 aprobadas, 51 casos válidos, 56 errores esperados y 0 fallos reales.



Figura 13: Categoría *Generación de código intermedio*: 12/12.

La batería también puede ejecutarse desde la línea de comandos:

```
ctest --test-dir build/compiscript --output-on-failure
```

Listing 5: Ejecución desde CTest.

12. Compilación y ejecución

12.1. Requisitos

- CMake 3.20 o superior.
- Compilador compatible con C++17.
- Java, únicamente para ejecutar la herramienta generadora de ANTLR (el código generado ya está versionado en el repositorio).
- Python 3 con Flask, únicamente para servir el IDE.

12.2. Construcción

Un solo script descarga el JAR oficial de ANTLR si hace falta, regenera el lexer, el parser y el Visitor, descarga y compila el *runtime* C++ de ANTLR, construye `compiscript_cli` y `compiscript_tests`, y ejecuta la batería:

```
python3 -m pip install -r requirements.txt
./scripts/build_compiscript.sh
```

Listing 6: macOS o Linux.

```
python -m pip install -r requirements.txt
powershell -File scripts/build_compiscript.ps1
```

Listing 7: Windows PowerShell.

12.3. Ejecución del IDE

```
python app.py
```

Listing 8: Servidor del IDE.

El IDE queda disponible en `http://127.0.0.1:5050`. Se abre un ejemplo, se pulsa *Analizar programa* y se selecciona la pestaña *Código TAC*.

12.4. Uso directo del CLI

```
./compiscript_cli --file backend/examples/compiscript/codigo_intermedio.cps
./compiscript_cli --stdin program.cps < program.cps
```

Listing 9: Compilación desde la línea de comandos.

La salida es un único documento JSON con los bloques `tokens`, `tree`, `semantic` y `tac` (`text`, `instructions`, `activation_records`, `temporaries_*`). El código de salida es 0 cuando no hay errores, 1 cuando el programa contiene diagnósticos y 2 cuando el uso del CLI o el archivo de entrada no son válidos.

12.5. Verificación realizada para este informe

Para escribir este informe se compiló y probó la rama `Compiladores2-Proyecto2` en Windows 11 con GCC 14.2 (MinGW-w64), CMake 3.31 y Ninja, usando el código fuente del *runtime* de ANTLR 4.13.2 ya disponible localmente. Los resultados observados fueron:

- La construcción completó sus 154 pasos sin errores.
- `compiscript_tests`: **107/107 pruebas aprobadas**; CTest reportó 1 de 1 pruebas superadas.
- `compiscript_cli` sobre `codigo_intermedio.cps` terminó con código 0 y generó 61 instrucciones; sobre `errores_semanticos.cps` terminó con código 1.
- El IDE respondió en `http://127.0.0.1:5050` y todas las capturas de este documento provienen de esa ejecución.

13. Decisiones, supuestos y limitaciones

Las decisiones siguientes están documentadas en el repositorio y son las que un calificador necesita para interpretar el TAC.

- **Solo se genera TAC para programas válidos.** Léxico, sintaxis y semántica deben terminar sin errores; si no, el TAC se marca como omitido y no hay instrucciones.

- **Las etiquetas son simbólicas.** Todavía no son direcciones de máquina; su resolución corresponde a la fase de código objeto.
- **`switch` no tiene caída implícita.** Cada `case` salta al final, coherente con la regla del proyecto que reserva `break` para ciclos.
- **Orden de evaluación.** Los argumentos se evalúan de izquierda a derecha y `%this` se pasa antes de los argumentos explícitos de un método.
- **Las listas son referencias dinámicas.** `foreach` se baja a índice, `length`, comparación, lectura y salto.
- **`try/catch` se conserva con operaciones abstractas** porque el proyecto no define un ABI de excepciones.
- **El TAC es una representación intermedia, no un intérprete ni código objeto.** No se ejecuta; su corrección se valida estructuralmente con la batería de pruebas.
- **Frames abstractos.** Los tamaños (1, 4, 8 bytes) y la alineación a 8 bytes son supuestos del modelo, no de una arquitectura concreta.

Limitaciones conocidas.

- No hay optimización: no se pliegan constantes (`1 + 2` genera una suma), no se elimina código muerto del TAC y no hay asignación de registros.
- En una asignación a un atributo o a un elemento de lista, el generador carga primero el lado izquierdo para conservar su localización, lo que deja una lectura cuyo valor no se usa (por ejemplo, `%t0 = %this.total` antes de `%this.total = ...`).
- Las verificaciones de la batería son estructurales (presencia y orden de operaciones, métricas, *layout*); no se interpreta el TAC para comparar resultados de ejecución.

14. Correspondencia con la rúbrica

Componente	Pts	Evidencia en el proyecto
Diseño de CI	25	Lenguaje de cuádruplos con formas de dirección, conjunto de instrucciones y sintaxis textual (sección 6), ejemplos de traducción por construcción (sección 7) y supuestos (sección 13); documentado también en `docs/codigo-intermedio-tac.md`.
Generación de TAC	65	`TacGenerator` de 1 211 líneas: expresiones, control de flujo, funciones, recursión, closures, clases, objetos y listas; reciclaje de temporales (sección 8); batería de 107 escenarios con 12 específicos de TAC (sección 11).
Tabla de símbolos	10	Campos `size`, `offset`, `storage`, `lexical_depth` y `label`; registros de activación con enlaces de control y acceso, retorno, parámetros, locales y temporales (sección 9); visibles en el IDE.
Total	**100**	

Tabla 19: Evidencia por componente evaluado.

Respecto a los entregables: el repositorio conserva commits individuales por integrante (sección 4), la batería cubre casos exitosos y fallidos de la generación de código (sección 11), la arquitectura y las instrucciones de ejecución están documentadas (secciones 3 y 12, además de `docs/arquitectura-compiscript.md` y `README.md`), el lenguaje intermedio tiene su documento propio y el IDE es funcional (sección 10).

15. Conclusiones

1. Separar el análisis semántico y la generación de código en dos Visitors evitó duplicar la resolución de nombres: el puente `SemanticBindings` permite que el generador trabaje con identificadores de símbolo en lugar de texto.
2. Generar TAC solo para programas sin errores simplifica el generador y elimina la posibilidad de producir código parcial o engañoso para entradas inválidas; el IDE y la batería lo verifican explícitamente.
3. El pool de temporales por función demuestra que el número de temporales necesarios depende del máximo de valores simultáneamente vivos y no del tamaño del programa: el ejemplo integral usa 8 temporales creados y 7 reutilizaciones, con a lo sumo 3 vivos a la vez.
4. Extender la tabla de símbolos con tamaños, offsets y frames, en lugar de crear una estructura aparte, mantiene una única fuente de verdad que el IDE, las pruebas y las fases posteriores pueden consultar.
5. Las limitaciones identificadas (sin optimización, lectura redundante del lado izquierdo, TAC no ejecutable) delimitan con claridad qué corresponde a las fases siguientes: optimización y generación de código objeto.

16. Referencias

- Aho, A., Lam, M., Sethi, R. y Ullman, J. *Compilers: Principles, Techniques, and Tools*. 2.ª edición, 2006. Capítulo 6 (código intermedio, cuádruplos y traducción dirigida por sintaxis) y secciones 7.2 y 7.3 (registros de activación y acceso a datos no locales).
- Parr, T. *The Definitive ANTLR 4 Reference*. Documentación del *target* de C++: `https://github.com/antlr/antlr4`.
- Enunciado del proyecto: *Proyecto 2: Generación de Código Intermedio para Compiscript*, Construcción de Compiladores, septiembre de 2026.
- Repositorio del proyecto, rama `Compiladores2-Proyecto2`: `https://github.com/G1LB3T0/Proyecto1_Compis_Generador_Analizador_Lexico`.
- Documentación interna del repositorio, en la carpeta `docs/`: `codigo-intermedio-tac.md`, `arquitectura-compiscript.md` y `decisiones-compiscript.md`.